Laith Haj Hosin — Tasks (Frontend shell, Auth pages, Take interview, Results, Profile)

Purpose

You build the frontend foundation and the logged-in experience: app entry, router, layout, login/register, keeping the token, protecting routes, taking an interview with a timer, the result page, and the profile pages. You are also the **contract coordinator**: if anyone needs to change `docs/API-CONTRACT.md`, you organize the team PR.

Read first: `docs/SPECIFICATION.md` (sections 4, 10), `docs/API-CONTRACT.md` (sections 1, 2, 4, 7, 9), `docs/TEAM-OWNERSHIP.md`.

1. Files you own (only you edit these)

File	What goes in it
`frontend/src/main.tsx`	Mount the app with the router and auth provider
`frontend/src/App.tsx`	App shell using the layout
`frontend/src/app/router.tsx`	All routes from spec §10; protected wrapper for everything except `/login`, `/register`. Adham adds his own route lines (coordination §3)
`frontend/src/layout/AppLayout.tsx`	Page frame + navbar
`frontend/src/layout/Navbar.tsx`	Links: Interviews, Questions, Profile; shows display name;

File	What goes in it
	Logout
`frontend/src/features/auth/AuthContext.tsx`	Current user + token state from contract §9 localStorage keys; `login`, `register`, `logout`; listens for `preppressure:logout`
`frontend/src/features/auth/ProtectedRoute.tsx`	Redirect to `/login` when no token
`frontend/src/features/auth/LoginPage.tsx`	Login form, `INVALID_CREDENTIALS` message
`frontend/src/features/auth/RegisterPage.tsx`	Register form, field validation, `USERNAME_TAKEN` / `EMAIL_TAKEN` messages
`frontend/src/features/auth/api.ts`, `types.ts`, `mocks.ts`	Auth calls (`/auth/*`, `PATCH /users/me`)
`frontend/src/features/attempts/StartAttemptPage.tsx`	Route `/interviews/:id/take`: call `POST /attempts`, then go to `/attempts/:newId`
`frontend/src/features/attempts/TakeAttemptPage.tsx`	Route `/attempts/:id`: questions, inputs per type, save each answer, finish button, redirect to results when ended
`frontend/src/features/attempts/AnswerInput.tsx`	MC radio buttons / short-answer textarea / code-output textarea; code

File	What goes in it
	snippet shown as plain text, **no run button**
`frontend/src/features/attempts/Timer.tsx`	Countdown from `deadlineAt` (display only)
`frontend/src/features/attempts/ResultsPage.tsx`	Route `/attempts/:id/results`: score, percentage, readiness, per-question explanation
`frontend/src/features/attempts/api.ts`, `types.ts`, `mocks.ts`	Attempt calls (contract §7)
`frontend/src/features/profile/ProfilePage.tsx`	`/profile`: own info, edit display name/bio, stats, list of my attempts with links to results
`frontend/src/features/profile/UserProfilePage.tsx`	`/users/:id`: public info + stats of another user
`frontend/src/features/profile/StatsCards.tsx`	Stats display used by both profile pages
`frontend/src/features/profile/api.ts`, `types.ts`, `mocks.ts`	`GET /users/:id/profile`, `GET /attempts`
`frontend/.env.example`	`VITE_API_URL`, `VITE_USE_MOCKS`
`README.md`	Day 1: create the file with all section headings (coordination §6). Then only *Overview*, *Project structure*, *Frontend setup*

2. Files you must NOT edit

- `frontend/src/features/questions/**`, `frontend/src/features/interviews/**` (Adham). Adham adds his own route lines to your router; you don't write them for him.
- Anything in `backend/`.
- Don't import from Adham's folders, and he doesn't import from yours. Links between pages are just URLs.

3. Implementation order (no one to wait for)

1. Day 1: commit `docs/API-CONTRACT.md`, `docs/SPECIFICATION.md`, `docs/TEAM-OWNERSHIP.md` exactly as delivered, plus the README skeleton.
2. `main.tsx`, `App.tsx`, router with your routes, layout, navbar.
3. Auth context + protected route + login/register using mocks (`VITE_USE_MOCKS=true`).
4. Take-interview flow with mock attempts: start → answer each type → timer → finish → results.
5. Results page, then profile pages.
6. Handle loading, empty and error states on every page (show the contract error `code` as a friendly message).
7. Integration day: switch mocks off and test against the real backend.

4. Things to think about

- Where does a page get the token? (Only from localStorage keys in contract §9 — not from another feature's code.)
- The timer reaches zero but the backend allows +5 s. What should the UI do at zero, and what if the backend still says `in_progress`?
- User reloads the page in the middle of an attempt: how do you show which questions are already answered? (Hint: `answeredQuestionIds`.)
- An answer save fails with `ANSWER_LATE`: what should the user see next?

5. Completion checklist

- [ ] Without a token, every page except login/register redirects to `/login`.
- [ ] Any 401 logs the user out (event from contract §9 works for all features, including Adham's).
- [ ] All three question types can be answered; code is never executed.
- [ ] Timer is driven by `deadlineAt`; timeout handled gracefully.
- [ ] Duplicate/late answer errors shown clearly.
- [ ] Results page only after the attempt ends; explanations readable.
- [ ] Profile shows correct stats; own profile can edit display name/bio; own attempts link to results.
- [ ] Works with mocks and with the real API; no TypeScript errors.